1. Introduction

The **Tower of Hanoi Game** is a Java-based interactive puzzle game developed as a project for the **Software Development I** course. The project is based on the classic Tower of Hanoi puzzle, where the main objective is to move all disks from the source tower to the destination tower while following the rules of the puzzle.

The application was developed using **Java Swing** for the graphical user interface. Object-Oriented Programming concepts were used to organize the different components of the system. The project also uses the **Stack data structure** to manage the disks and a **recursive algorithm** for the automatic solving feature. The project was developed incrementally, with different functionalities implemented and tested during different development weeks. GitHub was used to store the source code, documentation and track project progress.

---

2. Project Overview

The Tower of Hanoi Game contains three towers and three disks of different sizes.

Initially, all disks are placed on **Tower A**. **Tower B** is used as an auxiliary tower and **Tower C** is the destination tower.

The player needs to move all disks from Tower A to Tower C while following the rules of the Tower of Hanoi puzzle.

The application provides two main ways to solve the puzzle:

- **Manual Game:** The user manually selects the source and destination towers and moves the disks.
- **Automatic Solver:** The system automatically solves the puzzle using recursion.

The system also provides move validation, move counting, timer, restart functionality and winning detection.

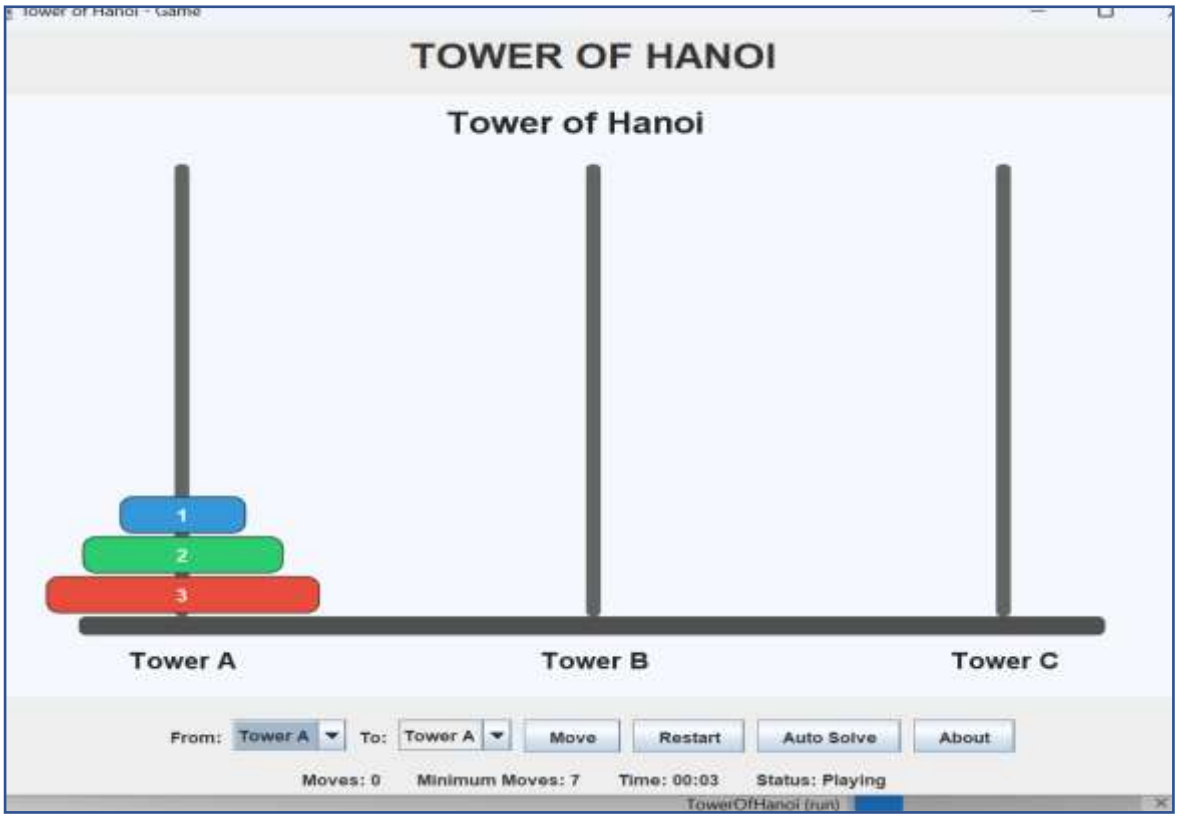


Figure 1: Main Game Interface

3. Project Objectives

The main objectives of the project are:

- To develop an interactive Tower of Hanoi game using Java.
- To create a simple and user-friendly graphical user interface.
- To allow users to move disks between three towers.
- To validate disk movements according to the game rules.
- To prevent illegal disk movements.
- To count the number of valid moves.
- To display the elapsed playing time.
- To provide a restart option.
- To detect successful completion of the puzzle.
- To implement an automatic solver using recursion.
- To apply Object-Oriented Programming concepts.
- To use GitHub for source code management and project progress tracking.

---

4. Development Methodology

The project was developed using the **Incremental Development Model**. In this methodology, the complete software was divided into smaller development increments. Each increment focused on a specific part of the system. After completing and testing one part, the next functionality was added. This approach made it easier to develop, test and improve the project step by step.

---

5. Development Progress

The project was developed over ten weeks.

Week	Main Work
Week 1	Project planning and requirement analysis
Week 2	System design, Class Diagram and Workflow Diagram
Week 3	Main Menu and basic GUI development
Week 4	Game Board, Tower and Disk structure
Week 5	Gameplay implementation and disk movement
Week 6	Rule validation and illegal move prevention
Week 7	Timer, move counter and restart functionality
Week 8	Automatic solver using recursion
Week 9	Win detection, success message and UI finalization
Week 10	Final testing, documentation and project release

---

6. System Requirements

6.1 Hardware Requirements

The application does not require high-end hardware.
Recommended hardware:

- Processor: Intel Core i5
- RAM: 8 GB
- Storage: 512 MB GB

- Keyboard and mouse
- Monitor

6.2 Software Requirements

The following software is required:

- Windows operating system
- Java Development Kit (JDK)
- NetBeans IDE
- GitHub account or access to the project repository

---

7. Project Features

The major features of the Tower of Hanoi Game are:

7.1 Main Menu

The application provides a main menu with options such as:

- Start Game
- Help
- Exit

7.2 Three Towers

The game contains:

- Tower A
- Tower B
- Tower C

7.3 Disk Movement

The user can select a source tower and a destination tower to move the top disk.

7.4 Move Validation

The system checks whether the selected movement follows the rules of the game.

7.5 Move Counter

The system counts the number of valid moves.

7.6 Timer

The timer records the time spent during gameplay.

7.7 Restart

The user can restart the game and return it to the initial state.

7.8 Automatic Solver

The system can automatically solve the puzzle using recursion.

7.9 Win Detection

The system detects when all disks reach the destination tower and displays a completion message.

---

8. System Architecture

The project follows a modular architecture. Each class is responsible for a specific part of the system.

Main Classes

TowerOfHanoi.java

Starts the application.

MainMenu.java

Displays the main menu and navigation options.

GameBoard.java
Displays the game interface, towers, disks, buttons, timer and game information.

GameController.java
Controls game logic, disk movement, validation, move counting and game completion.

Tower.java
Represents a tower and stores disks using a Stack.

Disk.java
Represents an individual disk and stores its size.

AutoSolver.java
Implements the recursive algorithm used for automatic solving.

---

9. Project Structure

The main project structure is:

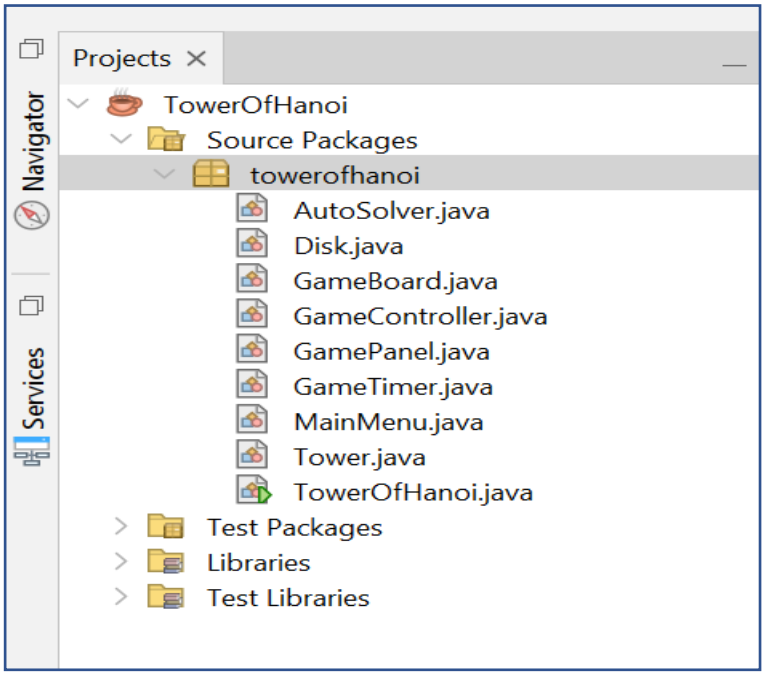


Figure 2: Project structure in NetBeans

---

10. Object-Oriented Programming Concepts

Several Object-Oriented Programming concepts were applied in this project.

10.1 Class and Object

The project contains different classes such as Disk, Tower, GameController and GameBoard. Objects are created from these classes to represent different components of the game.

10.2 Encapsulation

Data members are kept private and accessed through methods.

For example:

private int size;

The value can be accessed using:

public int getSize()

10.3 Abstraction

Different classes perform different responsibilities. For example, GameController handles game logic while GameBoard handles the graphical interface.

10.4 Modularity

The project is divided into multiple classes instead of keeping the complete program in one class. This makes the software easier to understand, maintain and modify.

---

11. Data Structure Used

The project uses the **Stack** data structure to store disks inside each tower.

A Stack follows the **LIFO (Last In, First Out)** principle.

The main operations used are:

- push() — adds a disk to the top.
- pop() — removes the top disk.
- peek() — checks the top disk without removing it.

Stack is suitable for the Tower of Hanoi because only the **top disk** of a tower can be moved.

---

12. Game Rules

The game follows the standard Tower of Hanoi rules:

1. Only one disk can be moved at a time.
2. Only the top disk of a tower can be moved.
3. A larger disk cannot be placed on a smaller disk.
4. All disks must be moved from Tower A to Tower C to complete the puzzle.

For three disks, the minimum number of moves is:

$$2^n - 1$$

Where:

$$n=3$$

Therefore:

$$2^3\text{-}1=8\text{-}1=2$$

So, the minimum number of moves required to solve the three-disk puzzle is **7 moves**.

---

13. Automatic Solver

The automatic solver uses the recursive Tower of Hanoi algorithm.

The recursive process is:

1. Move n-1 disks from the source tower to the auxiliary tower.
2. Move the largest disk from the source tower to the destination tower.
3. Move the n-1 disks from the auxiliary tower to the destination tower.

The recurrence relation is:

$$T(n)=2T(n-1)+1$$

For three disks:

$$T(1)=1$$

$$T(2)=2(1)+1=3$$

$$T(3)=2(3)+1=7$$

Therefore, the automatic solver can solve the three-disk puzzle in the minimum **7 moves**.

---

14. User Interface

The graphical user interface was developed using **Java Swing**.

The main interface contains:

- Main Menu
- Game Board
- Three Towers
- Disk representation
- Move button
- Restart button
- Auto Solve button
- Move counter
- Timer
- Game status/winning message

The GUI allows users to interact with the game without using command-line input.



Figure 3: Main Menu of the application

---

15. How to Download the Project from GitHub

A user can download the project from the GitHub repository.

Step 1: Open the GitHub Repository

The user first needs to open the project's GitHub repository.

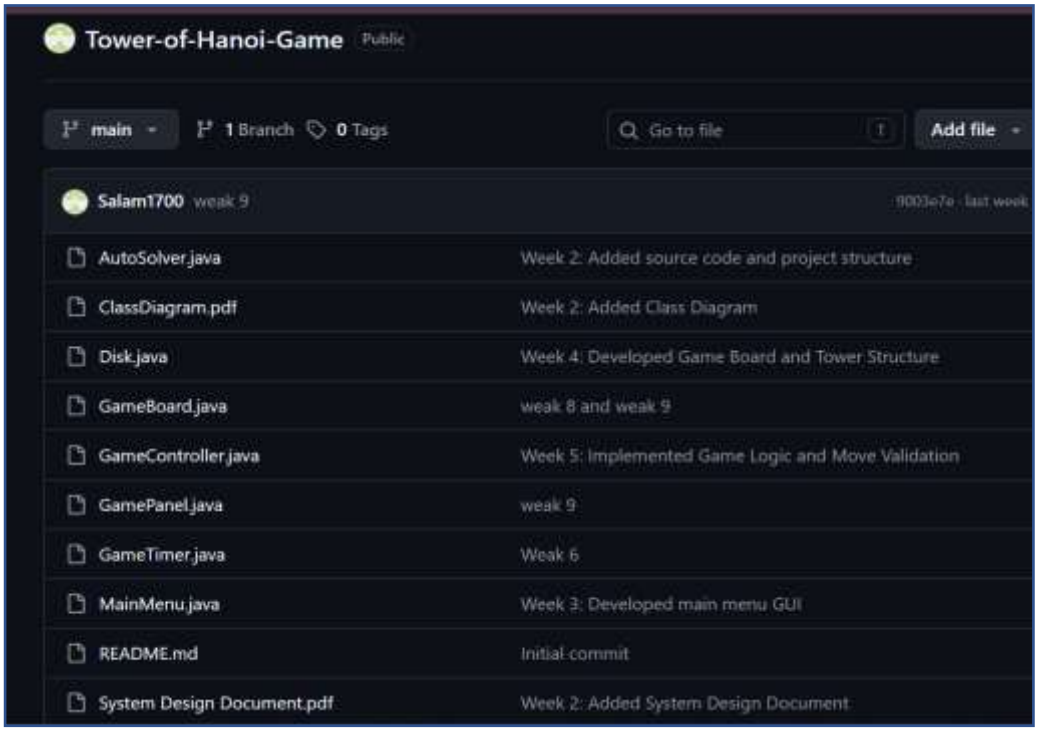


Figure 4: Tower of Hanoi Game GitHub repository

Step 2: Click the Code Button

After opening the repository, click the **Code** button.

Step 3: Select Download ZIP

From the Code menu, select **Download ZIP**.

GitHub will download the project as a ZIP file.

Figure 5: Downloading the Project



Figure 5: Downloading the project from GitHub

Step 4: Extract the ZIP File
After downloading, extract the ZIP file.
The extracted folder will contain the project files.

---

16. How to Open the Project in NetBeans
After extracting the project, the user can open it in NetBeans.
Steps:

1. Open **NetBeans IDE**.
2. Select **File → Open Project**.
3. Browse to the extracted project folder.
4. Select the Tower of Hanoi project.
5. Click **Open Project**.

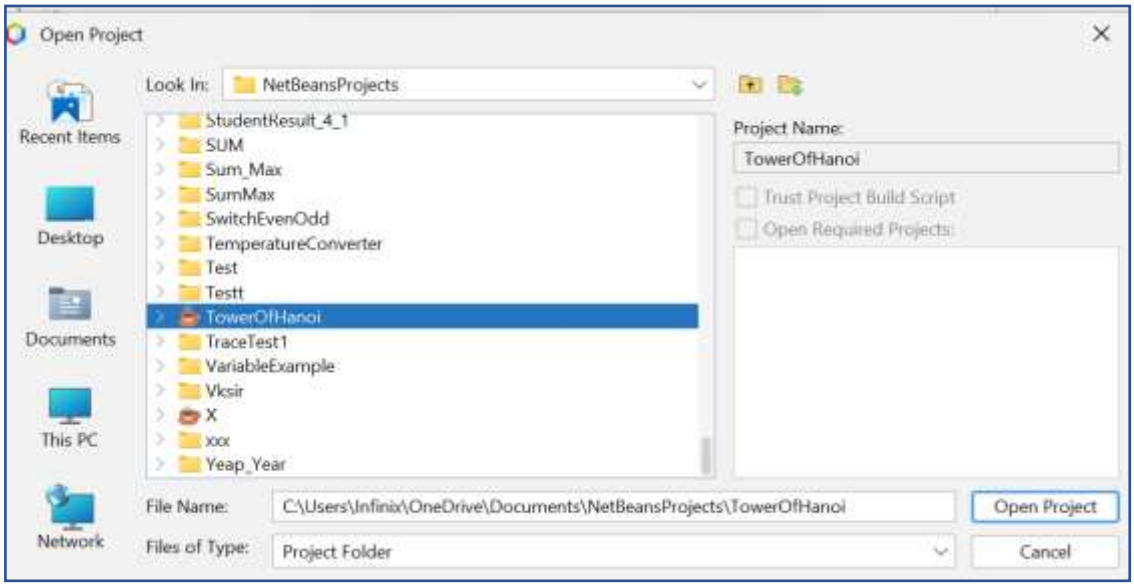


Figure 6: Opening the project in NetBeans

---

17. How to Run the Game
After opening the project in NetBeans:

1. Select the project.
2. Right-click the project.
3. Select **Run**.

The application will start and the Main Menu will appear.
The user can also use the **Run button** in NetBeans.

---

18. How to Use the Game
After starting the application:
Step 1
Select **Start Game** from the Main Menu.
Step 2
The Game Board will appear.

Step 3
Select the source tower from the **From** option.
Step 4
Select the destination tower from the **To** option.
Step 5
Click the **Move** button.
Step 6
The system checks whether the move is valid.

- If valid, the disk is moved.
- If invalid, the disk remains in its original tower and an appropriate message is displayed.



Figure 7: Disk movement during gameplay

---

19. Move Validation

The application validates every disk movement before performing it.
For example, if a larger disk is placed on a smaller disk, the system rejects the movement.
The system also rejects:

- Moving from an empty tower.
- Selecting the same tower as source and destination.
- Placing a larger disk on a smaller disk.



Figure 8: Invalid move validation

20. Timer and Move Counter

The application provides a timer and move counter.

Timer

The timer records the elapsed time during gameplay.

Move Counter

The move counter records the number of valid moves made by the user.

Invalid moves do not increase the move counter.



Figure 9: Timer and move counter

---

21. Restart Function

The **Restart** button allows the user to restart the puzzle.

After restarting:

- All disks return to Tower A.
- Tower B becomes empty.
- Tower C becomes empty.
- Move count is reset.
- The game starts from the initial state.

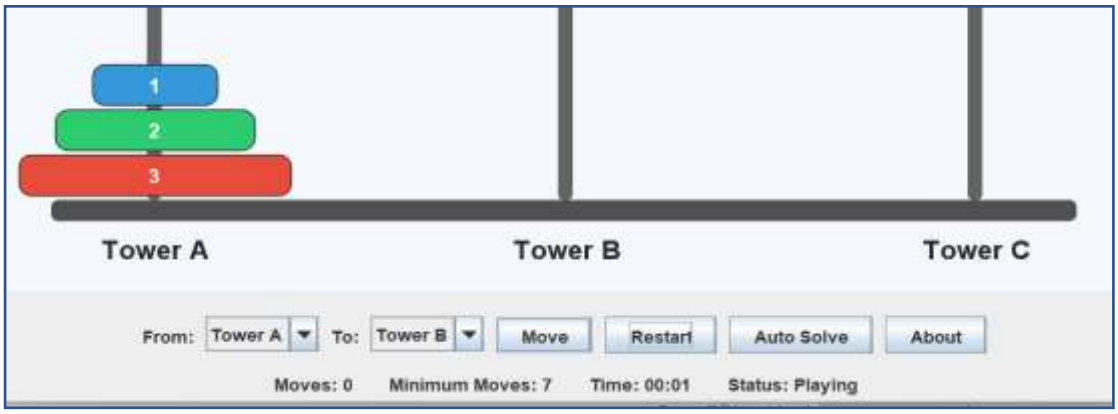


Figure 10: Restarting the game

---

22. Auto Solve Function

The **Auto Solve** button automatically solves the Tower of Hanoi puzzle.

The solver uses the recursive algorithm discussed earlier.

For three disks, the minimum solution requires:

$$2^3-1=7 \text{ moves.}$$

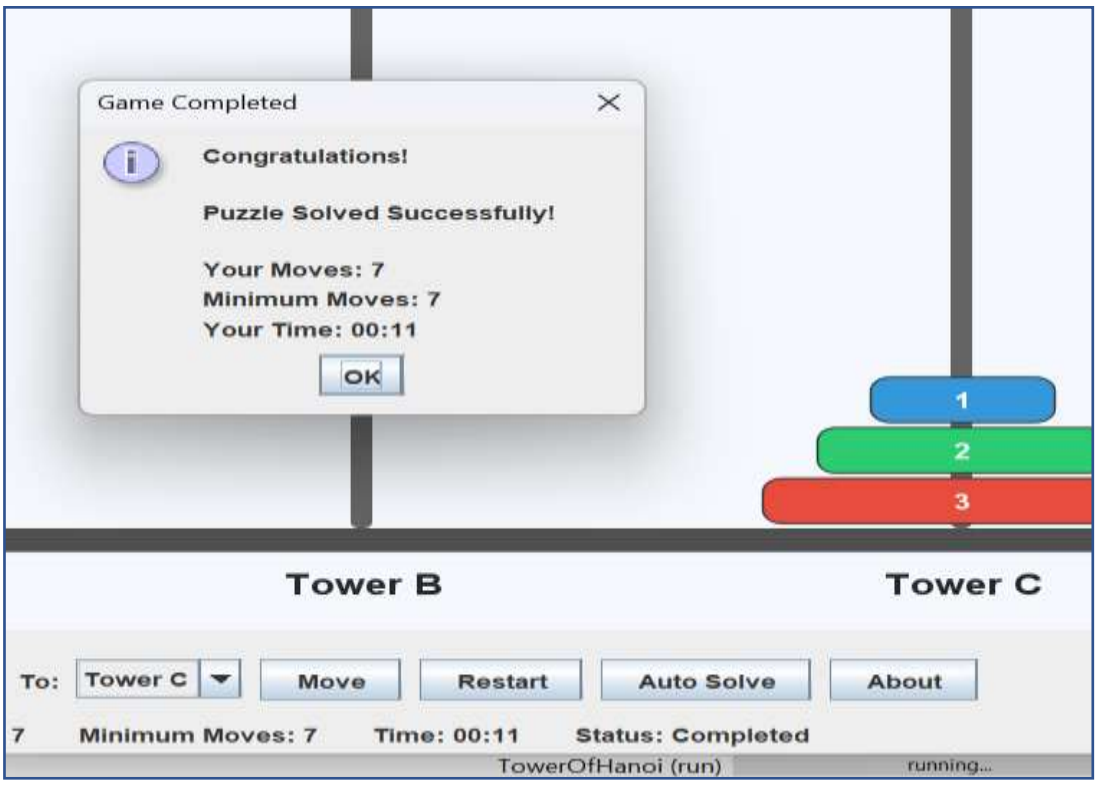


Figure 11: Automatic solving of the puzzle

23. Winning Condition

The user successfully completes the game when all disks are moved to **Tower C**.

After detecting the completed puzzle, the application displays a winning or completion message.

Figure 12: Winning Message

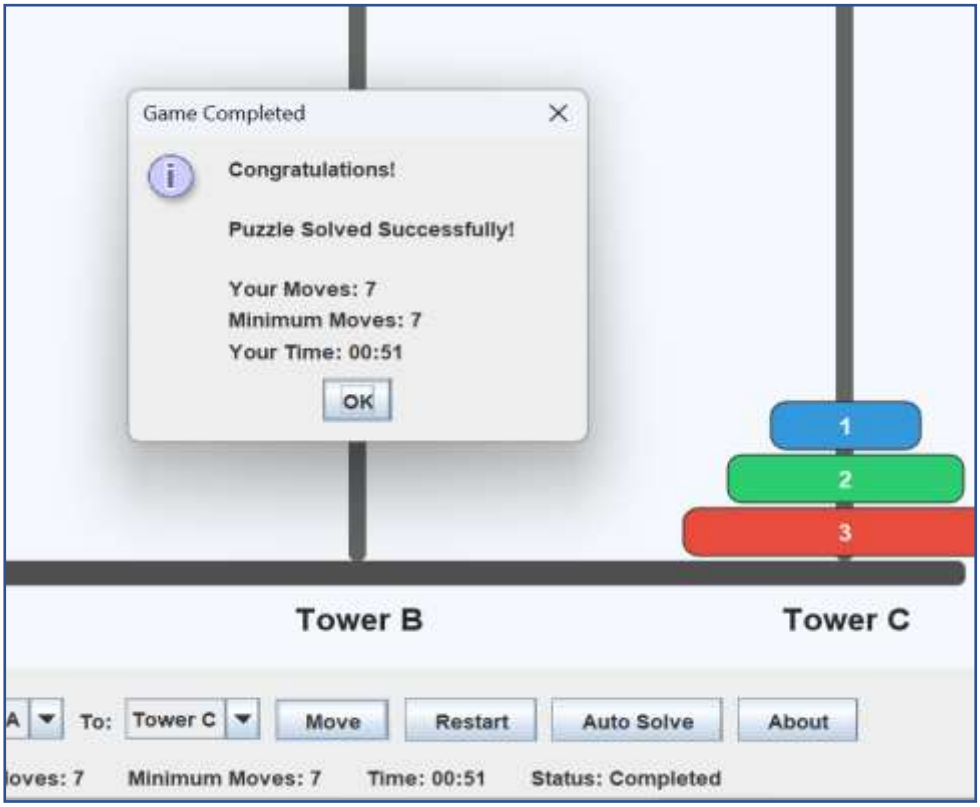


Figure 12: Successful completion of the game

24. Testing

The project was tested to verify the major functionalities.

Test Case	Expected Result	Status
Start Game	Game Board opens	Passed
Valid disk movement	Disk moves successfully	Passed
Larger disk on smaller disk	Move is rejected	Passed
Move from empty tower	Move is rejected	Passed
Same source and destination	Move is rejected	Passed
Valid move	Move counter increases	Passed
Restart Game	Game returns to initial state	Passed
Auto Solve	Puzzle is solved automatically	Passed
Complete puzzle	Winning message appears	Passed
Timer	Time increases during gameplay	Passed

Note: The test status should represent the actual tests performed on the final version of the application.

---

25. Error Handling and Validation

The system checks several conditions before performing a disk movement.
If the source and destination towers are the same, the movement is rejected.
If the source tower is empty, the movement is rejected.
If the destination tower contains a smaller disk, the movement is rejected.
Only valid movements increase the move counter.
These validations help maintain the rules of the Tower of Hanoi puzzle.

---

26. GitHub Version Control

GitHub was used to store and manage the project source code and documentation.
The project was developed incrementally, and development progress was uploaded to GitHub throughout the project.
The repository contains the source code, documentation and project-related files.
Separate commits were used to track development progress during different weeks.

---

27. Challenges Faced

Several challenges were faced during the development of the project:

- Understanding the recursive Tower of Hanoi algorithm.
- Connecting the graphical interface with game logic.
- Implementing disk movement.
- Implementing move validation.
- Managing disks using the Stack data structure.
- Updating the GUI after each move.
- Implementing the automatic solver.
- Handling timer and move counter functionality.

- Debugging errors during development.

These challenges were addressed through gradual development, testing and debugging.

---

28. Limitations

The current version of the project has some limitations:

- The game currently uses a fixed number of disks.
- There is no online multiplayer feature.
- Player scores are not permanently stored.
- There is no database.
- Advanced animations are limited.
- The application is designed as a desktop Java application.

---

29. Future Improvements

The project can be improved in the future by adding:

- Multiple difficulty levels.
- Different numbers of disks.
- Sound effects.
- Better disk animations.
- Scoreboard and leaderboard.
- Player profiles.
- Save and load functionality.
- Dark mode.
- Advanced visual effects.
- Player performance statistics.

---

30. Result

The Tower of Hanoi Game was successfully developed as a Java Swing-based desktop application. The system provides an interactive game board with three towers and disks. Users can manually move disks while following the game rules. The system validates movements, prevents illegal moves, counts valid moves, records playing time, provides restart functionality and detects successful completion.

The automatic solver was also implemented using recursion. The final project demonstrates the practical use of Java, Object-Oriented Programming, Stack data structure, GUI development, event handling, game logic and recursion.

---

31. Conclusion

The **Tower of Hanoi Game** project provided practical experience in Java software development and Object-Oriented Programming.

The project was developed using the Incremental Development Model. The system was built step by step, starting from project planning and GUI development and continuing with game logic, move validation, timer, automatic solving, testing and final improvements.

The application allows a user to download the project from GitHub, open it in NetBeans and run the game. The user can manually solve the puzzle, restart the game, monitor the timer and move count, or use the automatic solver.

The project successfully demonstrates how **Java Swing, Object-Oriented Programming, Stack data structure, event handling and recursion** can be combined to develop a complete interactive software application.

---

32. Project Demonstration

During the final project demonstration, the following features can be shown:

1. Launch the application.
2. Show the Main Menu.
3. Start a new game.
4. Show the three towers and disks.
5. Make a valid disk movement.
6. Demonstrate an invalid movement.
7. Show the move counter.
8. Show the timer.
9. Restart the game.
10. Use the Auto Solve feature.
11. Show the winning/completion message.
12. Show the GitHub repository.
13. Show how the project can be downloaded and opened in NetBeans.